

JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 16 OF 17 - STUDY STEP DSA-16

Greedy Algorithms: Recognition, Proofs, and Scheduling

Local Choices, Exchange Arguments, Counterexamples, and Interval Decisions

BY

Vinay Reddy Kalluri

Recognize when a local choice can be proved safe, construct counterexamples when it cannot, and implement sorting-driven greedy schedules.

GREEDY CHOICE | EXCHANGE PROOF | INTERVALS | SORTING | SCHEDULING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to DSA 15 - Graphs. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Greedy solutions are short to code and hard to justify. This volume centers the proof obligation, not the superficial loop.

Readiness map

Decision	Evidence
START HERE	A locally best choice may leave a compatible subproblem; Items can be ordered by a decisive endpoint or cost; The interviewer asks why dynamic programming is unnecessary; A tempting rule needs a counterexample.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-15; Sorting, comparators, graphs, and complexity.
MOVE ON WHEN	State a greedy-choice property; Use exchange and stays-ahead reasoning; Implement interval scheduling safely; Distinguish proven greedy logic from unsupported intuition.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 15 - Graphs	YOU ARE HERE DSA 16 - Greedy Algorithms	DSA 17 - Dynamic Programming
-----------------	---	------------------------------

A note from Vinay

A greedy choice needs a proof, not a confident intuition. State why one locally best decision can replace the first decision of an optimal solution without making the remainder worse.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Greedy Foundations: A Local Choice Needs a Global Proof	4
Chapter 2 - SDE-2 Greedy Algorithms: Recognition, Proof, and Boundaries	8
Chapter 3 - Essential Greedy Invariant Clinics	27
Chapter 4 - Realistic Greedy Interview Rounds	30
Chapter 5 - Greedy Proof and Counterexample Workshop	33
Chapter 6 - Proving Greedy Correctness	41
Chapter 7 - Greedy Algorithms Practice Lab	50
Chapter 8 - Greedy Algorithms Practice Lab Solutions	52
Chapter 9 - Executable Greedy Proof and Oracle Checks	54
Part II - Practice and Handoff	61

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Greedy Foundations: A Local Choice Needs a Global Proof

A greedy algorithm commits to one locally attractive choice and does not revisit it. Short code is not evidence of correctness. A greedy solution needs structure showing that an optimal solution can begin with the chosen decision.

Start with a choice sequence

For interval scheduling, the task is to select as many non-overlapping intervals as possible. Candidate rules include earliest start, shortest duration, fewest conflicts, and earliest finish. Only earliest finish has the standard proof for the unweighted objective.

Why earliest finish? It leaves the largest remaining timeline for all later intervals.

JAVA

```
record Interval(int start, int end) {}

static int maximumCompatible(List<Interval> input) {
    List<Interval> intervals = new ArrayList<>(input);
    intervals.sort(Comparator.comparingInt(Interval::end)
        .thenComparingInt(Interval::start));
    int selected = 0;
    int previousEnd = Integer.MIN_VALUE;
    for (Interval interval : intervals) {
        if (interval.start() >= previousEnd) {
            selected++;
            previousEnd = interval.end();
        }
    }
    return selected;
}
```

The endpoint comparison depends on the contract. For half-open meetings `[start, end)`, a meeting starting exactly at previous end is compatible. Closed intervals may require `start > previousEnd`.

Exchange argument from zero

Let g be the earliest-finishing available interval. Consider an optimal schedule whose first interval is o . Because g finishes no later than o , replacing o with g cannot invalidate any later selected interval. Therefore some optimal solution begins with g . After choosing it, the remaining problem has the same form.

That proof supplies both:

- greedy-choice property: an optimal solution can start with the greedy choice;
- optimal substructure: the remaining choices form the same problem after the chosen finish.

Counterexamples are a design tool

To reject "shortest interval first," construct one short interval placed so it blocks two compatible intervals. To reject "highest value first" for 0/1 knapsack, choose one valuable heavy item versus multiple lighter items with a better combined value.

A strong interview answer actively searches for a counterexample before committing to a greedy rule.

WHY IT WORKS | Common proof styles

Exchange

Transform an optimal solution to use the greedy choice without making it worse.

Stays ahead

After each prefix of decisions, prove the greedy partial solution is at least as good on a progress measure as any competitor. Minimum-jump frontier reasoning often uses this style.

Cut property

Show the lightest safe edge across a partition can belong to an optimal spanning tree.

Greedy choice plus induction

Prove a greedy first choice is safe, then apply the same argument to the smaller remaining problem.

WHY IT WORKS | Sort key is part of the proof

Many greedy algorithms appear as "sort, then scan." The sort key encodes the theorem. Changing it changes the algorithm. Tie behavior may affect determinism or even correctness when multiple fields control feasibility.

Use comparator composition rather than subtraction:

JAVA

```
Comparator<Job> order = Comparator.comparingInt(Job::deadline)
    .thenComparingInt(Job::duration)
    .thenComparing(Job::id);
```

Greedy versus dynamic programming

Ask:

- 1 Does a locally best choice remain safe for every valid suffix?
- 2 Can I exchange it into an optimal solution?
- 3 Does the future need only a small summary of the past, or does it need to compare competing states?
- 4 Can I produce a small counterexample?

Unweighted activity selection is greedy. Weighted interval scheduling generally needs DP because accepting an interval trades its value against many alternative compatible schedules. Fractional knapsack is greedy by value density; 0/1 knapsack is not.

Reachability greedy

In Jump Game, maintain the farthest reachable index while scanning only positions already reachable.

JAVA

```
static boolean canReachEnd(int[] jumps) {
    int farthest = 0;
    for (int index = 0; index < jumps.length && index <= farthest; index++) {
        farthest = Math.max(farthest, index + jumps[index]);
        if (farthest >= jumps.length - 1) return true;
    }
    return jumps.length == 0;
}
```

Invariant: every index through **farthest** is reachable using decisions from the processed reachable prefix. Use long or clamp arithmetic if index plus jump can overflow under the input contract.

Foundation failure clinic

- Choosing the most intuitive rule without proof.
- Giving only examples that work instead of a proof.
- Confusing interval merge with maximum compatible selection.
- Using value/weight density for indivisible 0/1 choices.
- Sorting caller-owned input without permission.
- Ignoring endpoint semantics and comparator overflow.
- Saying $O(n)$ while hiding $O(n \log n)$ sorting.

TRY IT | Foundation checkpoint

- 1 State the greedy choice for maximum unweighted interval selection.
- 2 Give the exchange step in one sentence.
- 3 Why does a successful example not prove greedy correctness?
- 4 Name a similar-looking problem where the same rule fails.
- 5 What part of a sort-and-scan solution carries the proof?

SDE-2 CORE CHAPTER

Chapter 2 - SDE-2 Greedy Algorithms: Recognition, Proof, and Boundaries

WHY IT WORKS | Why greedy needs a proof

A greedy algorithm commits to a locally preferred choice and does not revisit it. Sometimes that produces an optimal global solution; sometimes it produces a plausible wrong answer. The difference is not confidence or the number of examples tested. It is a structural proof showing that the local choice can belong to an optimum and that the remaining problem has the same form.

At SDE-2 level, name the choice rule, feasibility invariant, and proof method. If the proof fails, keep the greedy idea as a heuristic or switch to dynamic programming, search, or another exact algorithm. This chapter develops exchange, stays-ahead, and cut arguments across interval selection, merging, reachability, gas stations, scheduling, Huffman coding, fractional knapsack, and Dijkstra's boundary.

Learning objectives

After completing this chapter, you should be able to:

- recognize when local choice and optimal substructure may support a greedy solution;
- distinguish feasibility from optimality;
- prove a greedy choice with exchange, stays-ahead, or cut arguments;
- maximize compatible activities by earliest finishing time;
- merge interval coverage after sorting;
- solve Jump Game reachability and minimum jumps by frontier invariants;
- derive the gas-station restart rule and global feasibility test;
- choose scheduling orders for specific objectives rather than one universal rule;
- build optimal Huffman merge cost and fractional-knapsack value;
- explain why Dijkstra is greedy only under nonnegative weights; and
- produce counterexamples and a disciplined greedy-versus-DP decision.

Recognition checklist

A greedy approach is worth investigating when:

- the solution can be built as a sequence of irrevocable choices;
- choices have a natural ranking such as earliest finish, least cost, or best ratio;
- accepting one choice leaves a smaller problem of the same form;
- a locally chosen solution can be exchanged with the first choice of an optimum;
- one greedy partial solution can be shown to stay at least as good as any competitor; or
- a cut/property identifies one safe boundary-crossing choice.

Warning signals include:

- a choice consumes capacity and future value combinations matter;
- local scores do not account for interactions between choices;
- negative weights invalidate monotone progress;
- feasibility itself requires exploring alternatives; or
- a small counterexample defeats the proposed ordering.

Greedy is not "take the largest" or "sort and scan." The ranking must match the objective and admit a proof.

WHY IT WORKS | The proof toolkit

Exchange argument

Take an optimal solution **OPT**. Show that its first choice can be replaced by the greedy choice without making the solution infeasible or worse. After the exchange, some optimum begins greedily. Apply the argument to the remaining subproblem.

This proves activity selection: replacing the first activity of an optimum with the compatible activity that finishes earliest leaves at least as much time for everything afterward.

Stays-ahead argument

Compare the greedy partial solution with any competitor after every number of choices. Prove a measure of greedy progress is never behind. If it stays ahead to termination, no competitor achieves a better objective.

This proves the Jump Game frontier algorithm: after considering every index in the current jump layer, greedy has the farthest boundary reachable with that number of jumps.

Cut argument

Partition state into an accepted side and the remainder. Show that a best eligible item crossing the boundary is safe. Minimum spanning tree algorithms use the lightest cut edge. Dijkstra uses the smallest tentative-distance vertex, with nonnegative edges making it impossible for a later route across the cut to improve that distance.

Greedy-choice plus induction

After proving one safe choice, show the residual problem has the same structure, then apply induction. The proof must cover both the first choice and the recursive shape; optimal substructure alone does not prove the greedy choice.

Pattern 1: maximum compatible activities

Each activity is half-open interval $[start, end)$. Sort by increasing end time, then accept an activity when its start is at least the end of the last accepted activity.

Invariant: accepted activities are mutually compatible and, among schedules with the same accepted count drawn from the processed prefix, greedy's last finish time is no later than that of a competitor produced by exchanging choices.

Exchange proof

Let g be the earliest-finishing activity and o the first activity in an optimal schedule. Since $g.end \leq o.end$, replace o with g . Every later activity that started after o ended also starts after g ends. The schedule remains feasible with the same count. Therefore an optimum begins with g ; repeat on activities starting after $g.end$.

TRACE IT | Dry run

Activities $[1, 4), [3, 5), [0, 6), [5, 7), [3, 9), [5, 9), [6, 10), [8, 11), [8, 12), [2, 14), [12, 16)$ sort by finish. Greedy selects $[1, 4), [5, 7), [8, 11), [12, 16)$, count 4.

Sorting costs $O(n \log n)$ and scanning $O(n)$. If already sorted by finish, the scan is linear. Returning original IDs requires preserving them in records. Under half-open semantics, $[t, t)$ is empty: it is compatible with every activity, should always be retained when maximizing count, and must not advance the last occupied end time.

Counterexamples to tempting rules

- Earliest start picks $[0, 6)$ first and can lose shorter compatible activities.
- Shortest duration can pick an interval positioned in the middle that blocks good choices on both sides.
- Fewest conflicts is not the proven criterion and can be expensive to maintain.

The objective here is maximum count. Weighted interval scheduling, where activities have values, requires dynamic programming in general.

Pattern 2: merge interval coverage

Sort intervals by start, then end. Keep a current covered interval. If the next interval overlaps under the endpoint contract, extend the current end; otherwise emit the current interval and begin another.

Invariant: emitted intervals are sorted and disjoint and exactly cover all processed input except the current merged component. Because future intervals start no earlier than the current one, once a gap appears the emitted component can never be extended later.

For closed intervals $[1, 3]$, $[2, 6]$, $[8, 10]$, $[10, 12]$, output is $[1, 6]$, $[8, 12]$; touching at 10 merges. Under half-open interval semantics, $[8, 10)$ and $[10, 12)$ do not overlap unless product rules explicitly coalesce adjacency.

Time is $O(n \log n)$, space depends on sorting/copying and output. This greedy sweep computes the union; it is not the same objective as selecting the maximum number of nonoverlapping activities.

The implementation intentionally appears in both the arrays and greedy volumes. The arrays chapter uses it to teach defensive copying, nested-array ownership, and mutation safety; this chapter uses the same self-contained method to teach the sorted-gap invariant and greedy finalization proof. Keeping both lets each individual PDF remain runnable without importing source from another volume.

Pattern 3: Jump Game reachability

At index i , `jumps[i]` is the maximum forward length. Scan only indexes known reachable and maintain `farthest`, the greatest index reachable using some processed position.

Invariant before index i : every index up to `farthest` is reachable, and no processed index can reach beyond `farthest`. If $i > \text{farthest}$, there is a gap and the end is unreachable. Otherwise update `farthest = max(farthest, i + jumps[i])`.

Use `long` or a saturating comparison for $i + \text{jumps}[i]$ to avoid overflow when values are untrusted. Reject negative jump lengths under this model.

For $[2, 3, 1, 1, 4]$, start reach is 0. Index 0 extends to 2; index 1 extends to 4, reaching the end. For $[3, 2, 1, 0, 4]$, `farthest` becomes 3, but index 4 lies beyond it, so return false.

Time is $O(n)$, space $O(1)$. The proof is about the frontier of all processed choices, not choosing one specific next index.

Pattern 4: minimum jumps as BFS layers

When the end is reachable, maintain:

- `currentEnd`: farthest index reachable with the current number of jumps;
- `farthest`: farthest index reachable with one additional jump from positions in the current layer;
- `jumps`: number of completed layers.

Scan indexes before the last. Update `farthest`. When $i == \text{currentEnd}$, every position reachable with the current jump count has been considered; increment jumps and set `currentEnd = farthest`.

Stays-ahead invariant: after completing a layer, `currentEnd` is the maximum boundary any sequence using that many jumps can reach, because every possible predecessor in the prior reachable interval contributed its reach. Therefore the first layer reaching the end uses the minimum jumps.

For `[2, 3, 1, 1, 4]`, the first layer considers index 0 and ends at 2. The second layer considers indexes 1 and 2 and reaches 4. Answer: 2.

Detect a stalled layer (`farthest == currentEnd`) and return unreachable. Time $O(n)$, space $O(1)$.

Pattern 5: gas station circuit

At station `i`, gain `gas[i]` and spend `cost[i]` to reach the next station. Two facts produce the greedy solution.

- 1 If total gas is less than total cost, no start can complete the circuit.
- 2 If a tentative start accumulates a negative tank at index `i`, no station from that start through `i` can be a valid start; restart at `i + 1`.

Why can the whole failed segment be skipped? Before each intermediate station, the tentative tank was nonnegative. Starting later removes some nonnegative prefix contribution, so it cannot produce a better tank by the same failure point.

TRACE IT | Dry run

`gas=[1, 2, 3, 4, 5]`, `cost=[3, 4, 5, 1, 2]`. Deltas are `[-2, -2, -2, 3, 3]`. Starts 0,1,2 fail and reset. Starting at 3 accumulates 3,6, then wraps through negative deltas without dropping below zero. Total delta is zero, so start 3 works.

Time $O(n)$, space $O(1)$. Use `long` totals. Equal-length, nonempty arrays and nonnegative quantities are reasonable contracts. When multiple starts work, the scan returns one, not necessarily every start.

Pattern 6: scheduling objectives choose different rules

There is no universal greedy scheduling order.

Objective/model	Proven order
maximize number of compatible activities	earliest finish
minimize maximum lateness on one machine, all jobs available	earliest deadline
minimize average completion time, unweighted jobs	shortest processing time
minimize weighted completion sum	ratio/order derived by pairwise exchange
minimum rooms for fixed intervals	sort starts, track earliest end

Earliest deadline first for maximum lateness

A job has processing time p and deadline d . Completion time C gives lateness $C - d$; the objective minimizes maximum lateness. Sort by nondecreasing deadline.

Exchange proof: if adjacent jobs a, b are inverted with $d_a > d_b$, swap them. Earlier work before the pair is fixed. After swapping, b completes earlier and a completes at the original pair completion time. The maximum lateness of the pair does not increase. Repeatedly remove inversions to obtain deadline order.

This does not minimize missed-deadline count, weighted tardiness, or a multi-machine objective. Change the objective and the proof must be rebuilt.

Pattern 7: Huffman optimal merging

Given symbol frequencies, Huffman coding repeatedly combines the two least frequent subtrees. Their sum becomes a new subtree frequency inserted into a min-heap. Traversing left/right assigns prefix-free code bits; depth times frequency contributes to encoded length.

The merge cost is the sum of every combined frequency. For $[5, 9, 12, 13, 16, 45]$, combine $5+9=14$, $12+13=25$, $14+16=30$, $25+30=55$, $45+55=100$. Total cost is $14+25+30+55+100=224$ bits for the corresponding binary tree model.

Exchange idea: in some optimal prefix tree, the two least frequent symbols can be placed as deepest siblings. Combining them reduces the problem to a smaller optimal prefix-code instance. This justifies the repeated local merge.

Time is $O(n \log n)$, heap space $O(n)$. Ties permit multiple optimal trees; deterministic file formats need a tie-breaking and canonical-code specification. One symbol needs a format policy, often a one-bit code even though tree depth is zero.

The same rule minimizes total merge cost for files/ropes under additive merge cost.

Pattern 8: fractional knapsack

Items have value and positive weight; fractions are allowed. Sort by decreasing value-to-weight ratio and take as much as possible in that order.

Exchange proof: if a solution uses weight from a lower-ratio item while capacity could instead hold an equal weight from a higher-ratio item, exchanging that weight does not reduce value and usually improves it. Repeating exchanges yields greedy ratio order.

For capacity 50 and items $(value, weight) = (60, 10), (100, 20), (120, 30)$, ratios are 6, 5, 4. Take first two fully (weight 30, value 160), then 20/30 of the third (value 80), total 240.

Sorting costs $O(n \log n)$; scan $O(n)$. Validate finite nonnegative values and positive weights. Floating-point output tolerance must be defined. The reference comparator interprets each finite `double` through `BigDecimal.valueOf` and compares exact decimal cross-products, avoiding both primitive multiplication overflow and the collapse of distinct huge ratios to `Infinity`. A financial API should accept decimal values directly instead of first accepting binary floating-point inputs.

Critical counterexample: 0/1 knapsack

If fractions are forbidden, ratio greedy fails on the same items: it takes values 60 and 100 for 160, while items 100 and 120 fit exactly for 220. The 0/1 problem needs dynamic programming or another exact strategy under typical constraints. A tiny wording change invalidates the proof.

Dijkstra as a greedy boundary

Dijkstra repeatedly settles the unsettled vertex with smallest tentative distance. The cut is settled versus unsettled vertices. With nonnegative edges, any alternate path entering the unsettled side later cannot reduce that minimum: it starts at least as large and adds a nonnegative amount.

A negative edge destroys the cut argument. Example: edges $s \rightarrow a=2$, $s \rightarrow b=5$, $b \rightarrow a=-10$. Settling a at 2 is premature; path $s \rightarrow b \rightarrow a$ costs -5. Use Bellman-Ford, or topological relaxation for a DAG, when negative edges are allowed.

This illustrates a reusable habit: state the exact assumption used by the greedy proof. If input violates it, fail validation or choose another algorithm.

Greedy versus dynamic programming

Question	Greedy implication	DP implication
Can one optimal solution be exchanged to start with the local choice?	promising	if no, greedy unproved
Does one scalar frontier summarize all useful partial solutions?	promising	multiple incomparable states suggest DP
Does capacity couple current and future combinations?	often dangerous	state may include capacity
Are choices fractional/continuous?	ratio exchange may work	indivisible choices often need DP
Can a small adversarial example beat the ranking?	reject exact greedy	derive recurrence or search
Is approximate output acceptable?	heuristic may be valid with label	exact DP may be too expensive

When proposing greedy, actively search for counterexamples: equal scores with different future effects, one large choice versus several medium choices, negative values, and boundary ties. A heuristic may still be useful in production, but document approximation behavior and monitor quality.

Runnable Java 21 reference implementation

Run with `java -ea GreedyInterviewPatterns`.

JAVA (continued 1/8)

```
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public final class GreedyInterviewPatterns {
    private GreedyInterviewPatterns() {}

    public record Activity(String id, int start, int end) {
        public Activity {
            if (id == null || start > end) {
                throw new IllegalArgumentException("invalid activity");
            }
        }
    }

    public record Job(String id, int processingTime, int deadline) {
        public Job {
            if (id == null || processingTime < 0) {
                throw new IllegalArgumentException("invalid job");
            }
        }
    }

    public record ScheduleResult(List<String> order, long maximumLateness) {}

    public record FractionalItem(double value, double weight) {
        public FractionalItem {
            if (!Double.isFinite(value) || !Double.isFinite(weight)
                || value < 0 || weight <= 0) {
                throw new IllegalArgumentException("invalid item");
            }
        }
    }
}
```

JAVA (continued 2/8)

```

    }
}

public static List<Activity> selectMaximumActivities(List<Activity> activities) {
    if (activities == null || activities.stream().anyMatch(java.util.Objects::isNull)) {
        throw new IllegalArgumentException("activities must be non-null");
    }
    List<Activity> sorted = new ArrayList<>(activities);
    sorted.sort(Comparator.comparingInt(Activity::end)
        .thenComparingInt(Activity::start)
        .thenComparing(Activity::id));
    List<Activity> selected = new ArrayList<>();
    long lastEnd = Long.MIN_VALUE;
    for (Activity activity : sorted) {
        if (activity.start() == activity.end()) {
            selected.add(activity);
            continue;
        }
        if (activity.start() >= lastEnd) {
            selected.add(activity);
            lastEnd = activity.end();
        }
    }
    return List.copyOf(selected);
}

public static int[][] mergeClosedIntervals(int[][] intervals) {
    if (intervals == null) {
        throw new IllegalArgumentException("intervals must not be null");
    }
    int[][] copy = new int[intervals.length][2];
    for (int i = 0; i < intervals.length; i++) {
        if (intervals[i] == null || intervals[i].length != 2
            || intervals[i][0] > intervals[i][1]) {

```

JAVA (continued 3/8)

```
        throw new IllegalArgumentException("invalid interval");
    }
    copy[i] = intervals[i].clone();
}
Arrays.sort(copy, Comparator.comparingInt((int[] interval) -> interval[0])
    .thenComparingInt(interval -> interval[1]));
List<int[]> merged = new ArrayList<>();
for (int[] interval : copy) {
    if (merged.isEmpty() || interval[0] > merged.get(merged.size() - 1)[1]) {
        merged.add(interval.clone());
    } else {
        int[] last = merged.get(merged.size() - 1);
        last[1] = Math.max(last[1], interval[1]);
    }
}
return merged.toArray(int[][]::new);
}

public static boolean canReachEnd(int[] jumps) {
    validateJumps(jumps);
    if (jumps.length == 0) {
        return false;
    }
    long farthest = 0;
    for (int i = 0; i < jumps.length && i <= farthest; i++) {
        farthest = Math.max(farthest, (long) i + jumps[i]);
        if (farthest >= jumps.length - 1) {
            return true;
        }
    }
    return false;
}

public static int minimumJumps(int[] jumps) {
    validateJumps(jumps);
```

JAVA (continued 4/8)

```
    if (jumps.length <= 1) {
        return jumps.length == 1 ? 0 : -1;
    }
    long currentEnd = 0;
    long farthest = 0;
    int used = 0;
    for (int i = 0; i < jumps.length - 1 && i <= currentEnd; i++) {
        farthest = Math.max(farthest, (long) i + jumps[i]);
        if (i == currentEnd) {
            if (farthest == currentEnd) {
                return -1;
            }
            used++;
            currentEnd = farthest;
            if (currentEnd >= jumps.length - 1) {
                return used;
            }
        }
    }
    return -1;
}

public static int gasStationStart(int[] gas, int[] cost) {
    if (gas == null || cost == null || gas.length == 0 || gas.length != cost.length) {
        throw new IllegalArgumentException("equal nonempty arrays required");
    }
    long total = 0;
    long tank = 0;
    int candidate = 0;
    for (int i = 0; i < gas.length; i++) {
        if (gas[i] < 0 || cost[i] < 0) {
            throw new IllegalArgumentException("gas and cost must be nonnegative");
        }
        long delta = (long) gas[i] - cost[i];
        total += delta;
    }
}
```

JAVA (continued 5/8)

```
        tank += delta;
        if (tank < 0) {
            candidate = i + 1;
            tank = 0;
        }
    }
    return total >= 0 ? candidate % gas.length : -1;
}

public static ScheduleResult earliestDeadlineSchedule(List<Job> jobs) {
    if (jobs == null || jobs.stream().anyMatch(java.util.Objects::isNull)) {
        throw new IllegalArgumentException("jobs must be non-null");
    }
    List<Job> sorted = new ArrayList<>(jobs);
    sorted.sort(Comparator.comparingInt(Job::deadline).thenComparing(Job::id));
    List<String> order = new ArrayList<>();
    long time = 0;
    long maximumLateness = Long.MIN_VALUE;
    for (Job job : sorted) {
        time = Math.addExact(time, job.processingTime());
        maximumLateness = Math.max(maximumLateness, time - job.deadline());
        order.add(job.id());
    }
    if (sorted.isEmpty()) {
        maximumLateness = 0;
    }
    return new ScheduleResult(List.copyOf(order), maximumLateness);
}

public static long huffmanMergeCost(long[] frequencies) {
    if (frequencies == null) {
        throw new IllegalArgumentException("frequencies must not be null");
    }
    PriorityQueue<Long> queue = new PriorityQueue<>();
    for (long frequency : frequencies) {
```


JAVA (continued 6/8)

```
        if (frequency <= 0) {
            throw new IllegalArgumentException("frequencies must be positive");
        }
        queue.add(frequency);
    }
    long cost = 0;
    while (queue.size() > 1) {
        long merged = Math.addExact(queue.remove(), queue.remove());
        cost = Math.addExact(cost, merged);
        queue.add(merged);
    }
    return cost;
}

public static double fractionalKnapsack(FractionalItem[] items, double capacity) {
    if (items == null || !Double.isFinite(capacity) || capacity < 0) {
        throw new IllegalArgumentException("invalid items or capacity");
    }
    FractionalItem[] copy = items.clone();
    for (FractionalItem item : copy) {
        if (item == null) {
            throw new IllegalArgumentException("item must not be null");
        }
    }
    Arrays.sort(copy, GreedyInterviewPatterns::compareRatioDescending);
    double remaining = capacity;
    double value = 0;
    for (FractionalItem item : copy) {
        double taken = Math.min(remaining, item.weight());
        value += item.value() * (taken / item.weight());
        remaining -= taken;
        if (remaining == 0.0) {
            break;
        }
    }
}
```

JAVA (continued 7/8)

```
        return value;
    }

    private static int compareRatioDescending(FractionalItem first,
        FractionalItem second) {
        BigDecimal firstCross = BigDecimal.valueOf(first.value())
            .multiply(BigDecimal.valueOf(second.weight()));
        BigDecimal secondCross = BigDecimal.valueOf(second.value())
            .multiply(BigDecimal.valueOf(first.weight()));
        int ratioOrder = secondCross.compareTo(firstCross);
        if (ratioOrder != 0) {
            return ratioOrder;
        }
        int valueOrder = Double.compare(second.value(), first.value());
        if (valueOrder != 0) {
            return valueOrder;
        }
        return Double.compare(first.weight(), second.weight());
    }

    private static void validateJumps(int[] jumps) {
        if (jumps == null) {
            throw new IllegalArgumentException("jumps must not be null");
        }
        for (int jump : jumps) {
            if (jump < 0) {
                throw new IllegalArgumentException("jumps must be nonnegative");
            }
        }
    }

    public static void main(String[] args) {
        List<Activity> activities = List.of(
            new Activity("A", 1, 4), new Activity("B", 3, 5),
            new Activity("C", 0, 6), new Activity("D", 5, 7),
        );
    }
}
```

JAVA (continued 8/8)

```

        new Activity("E", 8, 11), new Activity("F", 12, 16));
    assert selectMaximumActivities(activities).stream().map(Activity::id).toList()
        .equals(List.of("A", "D", "E", "F"));
    assert selectMaximumActivities(List.of(
        new Activity("long", 0, 10), new Activity("empty", 5, 5))
        .stream().map(Activity::id).toList().equals(List.of("empty", "long"));
    assert Arrays.deepEquals(mergeClosedIntervals(
        new int[][] {{1, 3}, {2, 6}, {8, 10}, {10, 12}},
        new int[][] {{1, 6}, {8, 12}});

    assert canReachEnd(new int[] {2, 3, 1, 1, 4});
    assert !canReachEnd(new int[] {3, 2, 1, 0, 4});
    assert minimumJumps(new int[] {2, 3, 1, 1, 4}) == 2;
    assert minimumJumps(new int[] {3, 2, 1, 0, 4}) == -1;
    assert gasStationStart(new int[] {1, 2, 3, 4, 5},
        new int[] {3, 4, 5, 1, 2}) == 3;

    ScheduleResult schedule = earliestDeadlineSchedule(List.of(
        new Job("A", 3, 7), new Job("B", 2, 4), new Job("C", 1, 9)));
    assert schedule.order().equals(List.of("B", "A", "C"));
    assert schedule.maximumLateness() == -2;

    assert huffmanMergeCost(new long[] {5, 9, 12, 13, 16, 45}) == 224;
    double value = fractionalKnapsack(new FractionalItem[] {
        new FractionalItem(60, 10), new FractionalItem(100, 20),
        new FractionalItem(120, 30)}, 50);
    assert Math.abs(value - 240.0) < 1e-9;
    double hugeRatio = fractionalKnapsack(new FractionalItem[] {
        new FractionalItem(Double.MAX_VALUE / 2.0, Double.MIN_VALUE),
        new FractionalItem(Double.MAX_VALUE, Double.MIN_VALUE)},
        Double.MIN_VALUE);
    assert hugeRatio == Double.MAX_VALUE;
}
}

```

WHY IT WORKS | Complexity and proof table

Pattern	Time	Space	Proof core
activity selection	$O(n \log n)$	sorting/output dependent	exchange earliest finish
merge intervals	$O(n \log n)$	copy/output dependent	sorted gap finalizes coverage
jump reachability/min jumps	$O(n)$	$O(1)$	frontier stays ahead
gas station	$O(n)$	$O(1)$	failed segment cannot contain start
earliest-deadline schedule	$O(n \log n)$	$O(n)$ copy/output	adjacent inversion exchange
Huffman merge	$O(n \log n)$	$O(n)$	least frequencies as deepest siblings
fractional knapsack	$O(n \log n)$	$O(n)$ copy	exchange equal weight by ratio
Dijkstra	$O((V+E) \log V)$ typical heap form	graph/frontier	nonnegative cut safety

WATCH OUT | Edge cases and common mistakes

- 1 **A ranking without a proof.** Sorting by a plausible field does not establish optimality.
- 2 **Objective changed silently.** Maximum activity count differs from maximum value.
- 3 **Endpoint ambiguity.** Define whether touching activities are compatible and touching intervals merge.
- 4 **Jump overflow.** Widen `index + reach` before comparison.
- 5 **Empty Jump Game.** Decide whether no start means false; the toolkit returns false and minimum - 1.
- 6 **Gas total omitted.** Local restart logic alone cannot make an impossible circuit possible.
- 7 **One scheduling order reused.** Earliest deadline, shortest processing, and earliest finish solve different objectives.
- 8 **Huffman zero/negative frequency.** Remove zero-frequency symbols before construction and reject invalid counts.
- 9 **Nondeterministic Huffman ties.** Canonicalize codes when serialized compatibility matters.
- 10 **Fractional rule applied to 0/1 items.** Indivisibility destroys the exchange.
- 11 **Double ratio assumed exact.** Define numerical tolerance or use suitable exact decimals.
- 12 **Dijkstra with negative weights.** The greedy cut proof fails.
- 13 **Mutation hidden in sorting.** Copy caller inputs or document ownership transfer.
- 14 **Heuristic presented as exact.** Label approximation and quantify quality when proof is absent.

SDE-2 production follow-ups

- **Tie policy:** deterministic IDs after the primary greedy key make results reproducible without changing the proof when ties are equivalent.
- **Online arrivals:** offline sorted proofs may not survive streaming arrivals. Competitive analysis or reservation rules may replace offline optimality.
- **Cancellation:** schedules change when jobs cancel; maintain data structures and recompute decisions only if irrevocability permits.
- **Fairness:** throughput-optimal priority may starve large or low-priority jobs. Aging and quotas deliberately modify the objective.
- **SLAs:** maximum lateness, missed count, weighted tardiness, and percentile latency are different metrics. Choose one explicitly.
- **Numeric safety:** use `long` for cumulative time, gas, cost, and merge frequencies; use `BigInteger` or bounded validation beyond long range.
- **Input ownership:** sorting is mutation. Copy at API boundaries unless ownership transfer is explicit.
- **Auditing:** retain why each item was accepted, rejected, or delayed when business decisions require explanation.
- **Approximation:** NP-hard scheduling and packing may use greedy approximation. State approximation guarantees when known and monitor gap against small exact samples.
- **Changing priorities:** an object already in a heap must be reinserted or versioned; mutating priority fields violates ordering.

TRY IT | Exercises with model checkpoints

Exercise 1: erase overlapping intervals

Return the minimum intervals to remove so the rest do not overlap.

Model checkpoints: maximize retained compatible intervals by earliest finish; answer is `n - selected`; define endpoint compatibility; preserve original IDs if removals must be reported.

Exercise 2: partition labels

Partition a string so each symbol appears in at most one part.

Model checkpoints: precompute last occurrence; current segment end is farthest last occurrence among scanned symbols; when index reaches end, the segment is forced and can close; specify code-unit/alphabet contract.

Exercise 3: job sequencing with deadlines and profit

Unit-time jobs earn profit if completed by deadlines.

Model checkpoints: sort profits descending and place each job in the latest free slot no later than deadline; exchange intuition preserves earlier slots; DSU can find slots efficiently; reject this rule for arbitrary processing times.

Exercise 4: canonical Huffman codes

Produce deterministic bit codes from code lengths.

Model checkpoints: Huffman tree determines optimal lengths; sort by length then symbol; assign canonical incremented codes; single-symbol policy and maximum length belong to format; decoder validates oversubscribed/incomplete tables.

Exercise 5: counterexample workshop

Disprove value/weight greedy for 0/1 knapsack and earliest-start greedy for activity selection.

Model checkpoints: counterexample must satisfy all input constraints; compare complete feasible outputs; make it minimal enough to explain; identify exactly which exchange step fails.

Exercise 6: multi-objective scheduler

Design a scheduler balancing deadlines, fairness, and cost.

Model checkpoints: no single textbook order optimizes all objectives; define weights or lexicographic priorities; test starvation; consider an exact small-instance optimizer and a measured heuristic at scale; surface trade-offs to stakeholders.

Interview answer checklist

- ☐ I stated the exact objective and feasibility constraints.
- ☐ I named the local choice and the state retained after it.
- ☐ I supplied exchange, stays-ahead, cut, or induction reasoning.
- ☐ I tested tempting alternative rankings with counterexamples.
- ☐ I defined endpoint, tie, empty, unreachable, and numeric behavior.
- ☐ I did not transfer a fractional proof to indivisible choices.
- ☐ I identified the exact assumption behind Dijkstra or another greedy cut.
- ☐ I can explain when DP or approximation is necessary.
- ☐ I covered determinism, fairness, online arrivals, and ownership.

RECAP | Summary

Greedy algorithms are local commitments backed by global structure. Earliest finish admits an exchange for activity count. Sorted interval coverage finalizes components at gaps. Jump Game frontiers stay ahead; gas-station failures eliminate whole start segments. Earliest deadline minimizes maximum lateness under its specific model. Huffman repeatedly joins least frequencies, and fractional knapsack exchanges weight toward higher ratios. Dijkstra's cut is safe only with nonnegative edges. Counterexamples define the boundary: weighted intervals, 0/1 capacity, negative edges, and changed scheduling objectives need different reasoning. The SDE-2 standard is proof first, implementation second, and explicit operational trade-offs throughout.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Greedy Invariant Clinics

Greedy problems are not unified by sorting alone. Candy allocation and partition labels use different forms of irreversible information: local directional constraints and a farthest required boundary.

Clinic 1: candy allocation with two directional passes

Each child must receive at least one candy. A child with a higher rating than an adjacent child must receive more candy than that neighbor.

A left-to-right pass satisfies every rising constraint from the left. A right-to-left pass satisfies every rising constraint from the right. At each position, keep the maximum requirement from the two directions.

For ratings `[1, 0, 2]`:

- initial candies: `[1, 1, 1]`;
- left pass: `[1, 1, 2]`;
- right pass: `[2, 1, 2]`;
- answer: 5.

Why is it minimal? Each pass computes a lower bound forced by one neighbor direction. Taking their maximum is necessary to satisfy both, and the construction meets those bounds exactly.

Use `long` for the total when n can be large. A slope-counting solution can reduce auxiliary space to $O(1)$, but its peak accounting is easier to get wrong and should follow, not replace, the two-pass derivation.

Clinic 2: partition labels by closing obligations

Record the last position of each symbol. While scanning a partition, extend its required end to the farthest last occurrence of any symbol already seen. When the current index reaches that end, every symbol in the partition has fulfilled its future obligation, so the partition can close.

Closing earlier is impossible because at least one seen symbol would occur again outside. Closing exactly at the farthest obligation therefore yields the maximum number of valid partitions.

The code below defines symbols as Java UTF-16 `char` values. If the product contract means Unicode code points or user-perceived characters, iterate and index that representation explicitly.

Runnable Java 21 clinic

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Objects;

public final class GreedyCoverageClinic {
    private GreedyCoverageClinic() {}

    public static long minimumCandies(int[] ratings) {
        Objects.requireNonNull(ratings, "ratings");
        if (ratings.length == 0) {
            return 0;
        }
        int[] candies = new int[ratings.length];
        Arrays.fill(candies, 1);

        for (int index = 1; index < ratings.length; index++) {
            if (ratings[index] > ratings[index - 1]) {
                candies[index] = candies[index - 1] + 1;
            }
        }
        for (int index = ratings.length - 2; index >= 0; index--) {
            if (ratings[index] > ratings[index + 1]) {
                candies[index] = Math.max(candies[index], candies[index + 1] + 1);
            }
        }

        long total = 0;
        for (int candy : candies) {
            total += candy;
        }
    }
}
```

JAVA (continued 2/2)

```

        return total;
    }

    public static List<Integer> partitionLabels(String text) {
        Objects.requireNonNull(text, "text");
        int[] last = new int[Character.MAX_VALUE + 1];
        Arrays.fill(last, -1);
        for (int index = 0; index < text.length(); index++) {
            last[text.charAt(index)] = index;
        }

        List<Integer> lengths = new ArrayList<>();
        int start = 0;
        int requiredEnd = -1;
        for (int index = 0; index < text.length(); index++) {
            requiredEnd = Math.max(requiredEnd, last[text.charAt(index)]);
            if (index == requiredEnd) {
                lengths.add(index - start + 1);
                start = index + 1;
            }
        }
        return List.copyOf(lengths);
    }

    public static void main(String[] args) {
        assert minimumCandies(new int[] {1, 0, 2}) == 5;
        assert minimumCandies(new int[] {1, 2, 2}) == 4;
        assert partitionLabels("ababcbacadefegdehijhklij").equals(List.of(9, 7, 8));
        System.out.println("PASS essential greedy clinics");
    }
}

```

Expected output with assertions enabled:

TEXT

PASS essential greedy clinics

Interviewer follow-up chain with model answers

Interviewer: Why can candy not be solved by one left-to-right pass?

Candidate: A descending run imposes constraints from the right that are unknown during a left-only pass. The second pass supplies exactly those missing lower bounds.

Interviewer: Is partition labels an interval-merging problem?

Candidate: It can be viewed that way: each symbol spans first to last occurrence, and overlapping spans belong to one component. The one-pass farthest-end formulation performs that merge implicitly.

Interviewer: What should you say before presenting a greedy proof?

Candidate: State the exact objective and constraints, name the invariant or exchange, and show why no discarded decision can improve a later solution. A rule that merely works on examples is a heuristic, not yet a greedy proof.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Greedy Interview Rounds

Round 1: remove the fewest overlapping intervals

Prompt

Return the minimum number of half-open intervals to remove so the rest do not overlap.

Candidate derivation

Minimizing removals is equivalent to maximizing retained compatible intervals. Sort by end, retain each interval whose start is at least the last retained end, and remove the rest.

JAVA

```
static int minimumRemovals(int[][] input) {
    int[][] intervals = Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[1]));
    int kept = 0;
    int lastEnd = Integer.MIN_VALUE;
    for (int[] interval : intervals) {
        if (interval[0] >= lastEnd) {
            kept++;
            lastEnd = interval[1];
        }
    }
    return intervals.length - kept;
}
```

The exchange proof for activity selection applies. Time is $O(n \log n)$, auxiliary space depends on whether cloning/preserving input is required.

Follow-up

Weighted intervals? Earliest finish no longer maximizes value. Sort by end, locate previous compatible intervals, and use dynamic programming.

Streaming intervals? Without future knowledge, an irrevocable optimal offline selection is generally not available. State the online objective or approximation.

Round 2: gas station circuit

Prompt

Given gas gained and travel cost at each station, return a start index that completes the circular route, or -1.

Candidate answer

If total net gas is negative, no start works. While scanning, if the current candidate's running tank becomes negative at index i , no station between the candidate and i can succeed, so the next candidate is $i + 1$.

JAVA

```
static int startStation(int[] gas, int[] cost) {
    if (gas.length != cost.length || gas.length == 0) {
        throw new IllegalArgumentException("invalid arrays");
    }
    long total = 0L;
    long tank = 0L;
    int start = 0;
    for (int i = 0; i < gas.length; i++) {
        long net = (long) gas[i] - cost[i];
        total += net;
        tank += net;
        if (tank < 0L) {
            start = i + 1;
            tank = 0L;
        }
    }
    return total >= 0L ? start % gas.length : -1;
}
```

Follow-up answers

Why can skipped starts not work? They begin with no more accumulated surplus than the failed candidate had before reaching the same failing point, so each also runs negative by then.

Is the answer unique? Not necessarily. The method returns one valid start under the standard contract.

Round 3: minimum jumps to reach the end

Prompt

Each nonnegative array value is maximum jump length. Return the minimum number of jumps to reach the last index; return -1 if unreachable.

Candidate explanation

Interpret reachable indexes as BFS layers without storing a queue. `currentEnd` is the boundary reachable with the current number of jumps; `farthest` is the boundary reachable with one additional jump from the current layer.

JAVA

```
static int minimumJumps(int[] jumps) {
    if (jumps.length <= 1) return 0;
    int jumpsUsed = 0;
    int currentEnd = 0;
    int farthest = 0;
    for (int index = 0; index < jumps.length - 1; index++) {
        if (index > farthest) return -1;
        long reach = (long) index + jumps[index];
        farthest = (int) Math.min(jumps.length - 1, Math.max(farthest, reach));
        if (index == currentEnd) {
            jumpsUsed++;
            currentEnd = farthest;
            if (currentEnd >= jumps.length - 1) return jumpsUsed;
        }
    }
    return -1;
}
```

Follow-up answers

Proof intuition? All indexes through `currentEnd` form one BFS layer. Expanding every index in that layer finds the farthest boundary of the next layer. Advancing a layer once preserves minimum edge count.

Why not choose the largest immediate jump? The landing position's future reach matters. The layer method evaluates all positions reachable with the same jump count.

Closing answer pattern

State the local choice, rejected tempting choices, exchange/stays-ahead proof, endpoint and mutation contracts, sorting cost, numeric safety, and a nearby problem where greedy fails.

SDE-2 CORE CHAPTER

Chapter 5 - Greedy Proof and Counterexample Workshop

A greedy algorithm commits to a locally attractive choice without revisiting it. The loop is often short; the proof is the real solution. At SDE-2 level, "it seems optimal" is not enough. Name the choice, show why an optimal solution can include it, and state the invariant after committing.

The executable interval, jump, gas-station, and refueling implementations are in `GreedyInterviewChecks.java`.

Recognition: choice plus irreversible safety

A problem is a greedy candidate when:

- one decision can be ordered by a useful criterion;
- committing to the best current criterion leaves an equivalent smaller problem; and
- an exchange, cut, dominance, or frontier argument proves no optimum is lost.

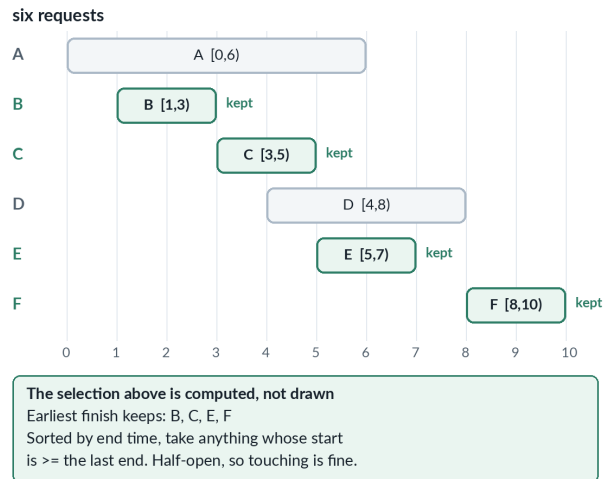
Greedy is suspicious when choices consume resources in ways that depend on future combinations. Coin change with arbitrary denominations is the standard warning: choosing the largest coin first fails for coins `[1, 3, 4]`, amount 6 (`4+1+1` versus optimal `3+3`).

Interval scheduling: earliest finish leaves most room

For maximum number of compatible half-open intervals $[start, end)$, sort by end and select an interval when $start \geq lastEnd$.

Why earliest finish wins, drawn

The exchange argument is one picture: swapping in the greedy choice never costs you anything



The exchange argument, in four lines

1. Greedy picks g , the earliest finisher.
2. Some optimal schedule starts with o .
3. $g.end \leq o.end$, so swapping g for o leaves every later interval still legal.
4. So an optimum exists containing g . Repeat on what remains.

Why the other two rules fail

earliest start $\rightarrow$ a long early request blocks everything
 shortest first $\rightarrow$ a short request in the middle splits the line

Both were run against an exhaustive oracle.
 Earliest finish was optimal in 500 of 500 random instances; the other two were not.

The exchange argument, with a computed selection

The highlighted intervals in that figure are not a drawing decision - the generator runs the earliest-finish rule on those six requests and marks whatever it selects. If the rule changed, the picture would change with it.

Exchange proof:

- 1 Let greedy choose interval g with earliest finish.
- 2 Let an optimal schedule begin with o .
- 3 Since $g.end \leq o.end$, replace o with g .
- 4 Every interval compatible after o is still compatible after g .
- 5 An optimum exists containing the greedy choice; repeat on the suffix.

The companion returns the selected original records and IDs, not only a count. Deterministic tie-breaks make the result auditable.

JAVA

```
import java.util.*;

record Interval(String id, int start, int end) { } // half-open [start, end)

final class IntervalScheduling {
    private static final Comparator<Interval> BY_FINISH =
        Comparator.comparingInt(Interval::end)
            .thenComparingInt(Interval::start)
            .thenComparing(Interval::id); // unique - see Chapter 30

    static List<Interval> selectMaximum(Collection<Interval> intervals) {
        List<Interval> sorted = new ArrayList<>(intervals);
        sorted.sort(BY_FINISH);

        List<Interval> chosen = new ArrayList<>();
        int lastEnd = Integer.MIN_VALUE;
        for (Interval candidate : sorted) {
            if (candidate.start() >= lastEnd) { // half-open: touching is fine
                chosen.add(candidate);
                lastEnd = candidate.end();
            }
        }
        return List.copyOf(chosen);
    }
}
```

The comparator ends on `id` for the reason Chapter 30 gives: without a unique final tie-break, two intervals with identical bounds order arbitrarily and the `set` returned changes between runs even though the count does not. A candidate who only returns a count never notices; one who returns the selection does.

`start() >= lastEnd` rather than `>` is the half-open convention doing real work - a meeting ending at 10:00 and one starting at 10:00 do not conflict. Getting this backwards costs one interval on exactly the inputs an interviewer constructs.

Sorting by earliest start is not safe: a very long early interval can block many short ones. A counterexample is part of explaining the correct criterion.

Minimum jumps as BFS layers without a queue

For each index, `farthest` is the greatest position reachable using one more jump from any index in the current layer. `currentLayerEnd` marks the end of positions reachable with the current number of jumps.

TEXT

```
values: [2,3,1,1,4]
layer 0 indexes: [0]      farthest becomes 2
layer 1 indexes: [1,2]    farthest becomes 4
destination reached in 2 jumps
```

When scanning reaches `currentLayerEnd`, commit one jump and extend the layer to `farthest`. If the scan index moves beyond `farthest`, the destination is unreachable.

The proof is the same as unweighted BFS: all indexes in a layer are reachable with the same minimum jump count, and the next layer is their union of outgoing ranges.

Gas station: discard an impossible prefix

Let $\text{net}[i] = \text{gas}[i] - \text{cost}[i]$. If total net is negative, no start can complete the circle. While scanning a candidate start, if running tank becomes negative at i , no station between that candidate and i can be a valid start: each would begin with no more accumulated surplus yet face the same failing suffix. Set candidate to $i+1$ and reset the local tank.

Use `long` for total and tank. Validate nonnegative gas/cost and equal nonempty arrays. The proof assumes every segment is traversed in fixed circular order.

JAVA

```
/** Index of the only possible starting station, or -1 if the circuit is impossible. */
static int startingStation(int[] gas, int[] cost) {
    if (gas.length != cost.length || gas.length == 0) {
        throw new IllegalArgumentException("gas and cost must be equal and non-empty");
    }
    long total = 0;
    for (int i = 0; i < gas.length; i++) {
        total += (long) gas[i] - cost[i];
    }
    if (total < 0) {
        return -1; // no start can finish; decided before scanning
    }

    int candidate = 0;
    long tank = 0;
    for (int i = 0; i < gas.length; i++) {
        tank += (long) gas[i] - cost[i];
        if (tank < 0) { // i is unreachable from `candidate`...
            candidate = i + 1; // ...and from every station between them
            tank = 0;
        }
    }
    return candidate;
}
```

This is one pass, not the $O(n^2)$ "try every start" the problem statement invites. The reason it works is the whole insight: when the tank goes negative at i , every station from `candidate` through i is eliminated at once, because each of them would reach i with no more surplus than `candidate` had. The scan therefore never revisits a station.

Note the two-part structure. The `total < 0` check answers *whether* a solution exists; the scan answers *where* it starts. Merging them is tempting and wrong - the scan alone cannot distinguish "no valid start" from "start at 0".

Checked against an exhaustive $O(n^2)$ simulation of every starting station over 4,000 random instances: identical answers, including the `-1` cases.

Refueling: defer the choice until necessary

As current fuel makes stations reachable, add their fuel amounts to a max-heap but do not stop immediately. When the target is not reachable, retroactively choose the largest fuel among all passed stations.

Why this is safe: every passed station was reachable before the current failure. Replacing any smaller chosen refill with the largest passed refill cannot reduce reach and uses the same one stop.

TEXT

```
reachable frontier -> collect station fuels  
frontier cannot advance -> take largest collected fuel  
repeat or report impossible
```

This produces the minimum number of stops, not minimum fuel cost. Adding prices, tank capacity, mandatory stops, or time windows changes the problem and can invalidate the greedy rule.

Test a greedy algorithm against an oracle

Proof is primary, but exhaustive small cases catch implementation bugs and false intuitions.

The companion:

- enumerates every interval subset for small random instances and compares maximum cardinality; and
- computes minimum jumps with a small dynamic-programming oracle and compares the greedy frontier result.

An oracle disagreement does not automatically prove which implementation is wrong, but it gives a reproducible counterexample for reasoning.

The oracle is worth writing once and keeping, because it is short:

JAVA

```

static int bruteForceMaximum(List<Interval> all) {
    int n = all.size();
    int best = 0;
    for (int mask = 0; mask < (1 << n); mask++) {    // every subset
        List<Interval> subset = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                subset.add(all.get(i));
            }
        }
        if (pairwiseCompatible(subset)) {
            best = Math.max(best, subset.size());
        }
    }
    return best;
}

private static boolean pairwiseCompatible(List<Interval> subset) {
    for (int i = 0; i < subset.size(); i++) {
        for (int j = i + 1; j < subset.size(); j++) {
            Interval a = subset.get(i);
            Interval b = subset.get(j);
            boolean disjoint = a.end() <= b.start() || b.end() <= a.start();
            if (!disjoint) {
                return false;
            }
        }
    }
    return true;
}

```

$1 \ll n$ limits this to roughly $n \leq 20$, which is the point: an oracle is allowed to be exponential because it only ever sees inputs small enough to enumerate. Its job is to be *obviously* correct, not fast.

Run over 500 random instances of up to seven intervals, earliest-finish matched the exhaustive optimum every time. The same harness is what shows the plausible alternatives failing - earliest-*start* and shortest-first each produce strictly worse answers on inputs this test finds within seconds.

Greedy versus dynamic programming

Signal	Greedy	Dynamic programming
decision can be safely exchanged into an optimum	strong candidate	may be unnecessary
choice affects several future resource dimensions	risky	model dimensions as state
need only current frontier summary	often	sometimes space-optimized DP
arbitrary coin denominations	generally fails	standard DP
interval maximum count	earliest-finish proof	possible but excessive
weighted interval profit	earliest finish no longer enough	DP after predecessor search

Do not choose DP merely because it feels safer; do not choose greedy merely because it is shorter.

Edge-case matrix

Case	Correct handling	Common failure
touching intervals	define half-open/closed semantics	inconsistent $\geq$ versus $>$
invalid interval	reject before sorting/selection	negative-duration reasoning
equal finish times	deterministic safe tie policy	nondeterministic outputs
empty interval list	empty selected list	sentinel counted as interval
empty/single jump array	zero jumps by stated contract	indexing position zero
unreachable jump prefix	return explicit failure	increment jumps forever
huge jump length	compute reach in <code>long</code> , clamp	index addition overflow
total gas negative	return impossible	returning reset candidate
target already fueled	zero stops	consuming a station anyway
zero-fuel station	harmless but cannot advance	infinite refuel loop
unsorted stations	reject or sort a copy	missing reachable stations
accumulated fuel overflow	clamp at target	wrapped negative reach

Six live interview Q&A chains

1. Interval criterion

Interviewer: Why not choose the shortest interval?

Candidate: Duration alone does not determine how much future timeline remains. Earliest finish has an exchange proof: replacing an optimal first interval with it cannot block a later compatible interval.

2. Weighted intervals

Interviewer: What if intervals have profit?

Candidate: Maximum count's earliest-finish proof no longer optimizes profit. I would sort by finish, binary-search the predecessor, and use DP over take/skip profit.

3. Jump frontier

Interviewer: Why increment jumps only at the layer end?

Candidate: Every index up to that boundary is reachable with the same minimum jump count. I first aggregate all their next reach, then one committed jump moves to that complete next layer.

4. Gas reset

Interviewer: Why can you discard every start between the candidate and failure?

Candidate: Starting later removes a prefix whose cumulative tank was nonnegative before eventual failure; it cannot give more tank at the failing edge than the original candidate. None can cross it.

5. Refueling choice

Interviewer: Are you pretending we stopped at an earlier station after passing it?

Candidate: The heap is an equivalent deferred-decision model. All stored stations were reachable. When a stop becomes necessary, choosing the largest one among them can be exchanged with any smaller prior choice without increasing stop count.

6. Proof confidence

Interviewer: Your randomized oracle passed. Is the greedy algorithm proven?

Candidate: No. Tests sample finite cases and catch bugs; the exchange/frontier argument establishes general correctness. I use both: proof for the algorithm, oracle for the implementation.

Run the companion

BASH

```
javac --release 21 -Xlint:all -Werror GreedyInterviewChecks.java
java GreedyInterviewChecks
```

Expected final line: **PASS 15 greedy checks.**

SDE-2 CORE CHAPTER

Chapter 6 - Proving Greedy Correctness

Why this chapter exists

Greedy is the only major algorithmic technique where a **plausible algorithm is usually wrong**, and where the difference between a passing and a failing interview answer is whether you can say *why* yours is right.

Dynamic programming considers every option, so a correct recurrence is correct by construction. Greedy commits to one choice and never reconsiders, so it needs an argument that the committed choice is safe. Without that argument you have a heuristic that happens to pass the examples.

The workshop chapter in this volume covers finding counterexamples - proving an algorithm *wrong*. This chapter covers the other direction: the two standard techniques for proving one *right*, and how to deploy them in the five minutes an interview allows.

WHY IT WORKS | The two proof techniques

Almost every greedy correctness argument is one of these.

Exchange argument

Claim: any optimal solution can be transformed, step by step, into the greedy solution without ever getting worse. Therefore greedy is at least as good as optimal, so it is optimal.

The structure is always the same:

- 1 Let **OPT** be an optimal solution and **G** the greedy one.
- 2 Find the first position where they differ.
- 3 Show that exchanging **OPT**'s choice for greedy's choice yields a solution that is still valid and no worse.
- 4 Repeat. Each exchange increases the agreement with **G** by one, so after finitely many steps **OPT** has become **G**.

Worked example - interval scheduling. Given intervals, select the maximum number that do not overlap. Greedy: sort by *earliest finish time*, take each interval compatible with those already chosen.

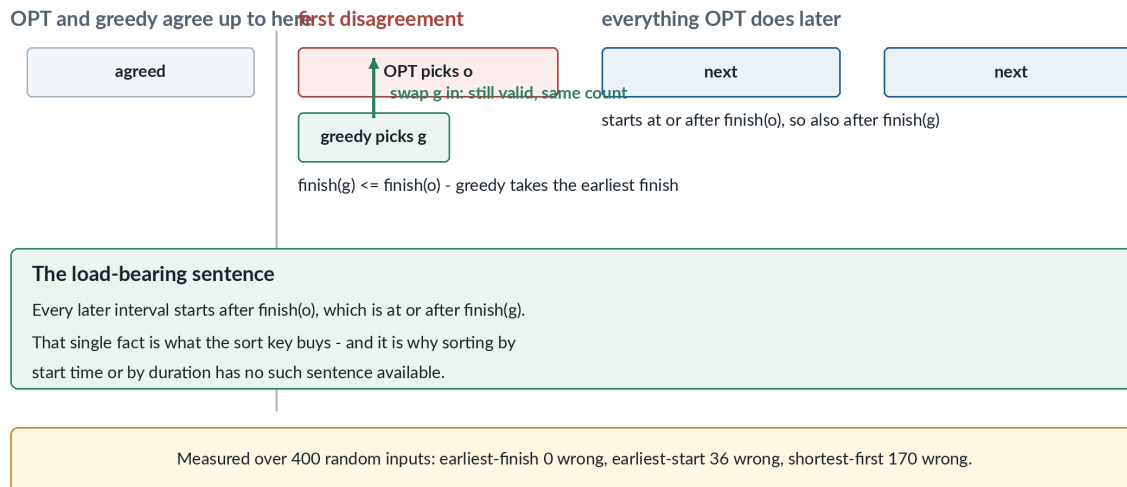
Proof. Let **OPT** be optimal, sorted by finish time, and **G** be greedy's selection. Suppose they agree on the first k intervals and differ at position $k+1$. Greedy chose **g** and **OPT** chose **o**. By greedy's rule, **g** has the earliest finish time among all intervals compatible with the first k , so $\text{finish}(\mathbf{g}) \leq \text{finish}(\mathbf{o})$.

Replace **o** with **g** in **OPT**. Is the result still valid? Every interval in **OPT** after position $k+1$ starts at or after $\text{finish}(\mathbf{o}) \geq \text{finish}(\mathbf{g})$, so none of them conflicts with **g**. The count is unchanged, so the modified solution is still optimal and now agrees with **G** on $k+1$ intervals.

Repeating drives the disagreement to zero. Greedy is optimal. **QED**

The exchange argument, in one picture

Greedy's interval finishes no later, so swapping it in never breaks validity



Java SDE-2 DSA Series | Volume 16: Greedy | Figure 01

The exchange step: greedy's earlier finish time never breaks a later interval

The load-bearing sentence is "every later interval starts after `finish(o)`, which is at least `finish(g)`." **That is where the sort key earns its correctness**, and it is the sentence an interviewer is waiting for. Sorting by start time, or by duration, has no such sentence available - which is exactly why those variants are wrong.

Greedy stays ahead

Claim: after every step, greedy's partial solution is at least as good as any other solution's partial solution by some measure. Therefore it is at least as good at the end.

This suits problems where you can define a running quantity and show greedy dominates it at each step.

Worked example - interval scheduling again, the other way. Let `g1..gk` be greedy's choices and `o1..om` any other valid selection, both by finish time.

Claim: for every $i \leq \min(k, m)$, $\text{finish}(g_i) \leq \text{finish}(o_i)$.

Induction. For $i = 1$, greedy picks the globally earliest finish, so it holds. Assume it holds for i . Then `o_{i+1}` starts at or after $\text{finish}(o_i) \geq \text{finish}(g_i)$, so `o_{i+1}` was available to greedy at step $i+1$. Greedy picks the earliest finishing available interval, so $\text{finish}(g_{i+1}) \leq \text{finish}(o_{i+1})$.

Conclusion. Suppose $m > k$. Then `o_{k+1}` exists and starts at or after $\text{finish}(o_k) \geq \text{finish}(g_k)$, so it was compatible with greedy's selection - contradicting greedy having stopped. Hence $m \leq k$. QED

Which to use

Use exchange when	Use stays-ahead when
The solution is a set or sequence of choices	There is a natural running measure
Swapping one choice for another is easy to justify	Progress is monotonic
Selection problems - scheduling, MST	Coverage problems - jump game, gas station

Both work for interval scheduling, which is why it is the standard teaching example. When only one is available, it is usually obvious which.

Applying this under time pressure

You will not write a formal proof in an interview. You will say two or three sentences, and they should be the load-bearing ones.

The template that works:

"I'll sort by X. The exchange argument is: take any optimal solution that differs from mine at the first position; my choice has [property P]; swapping it in keeps the solution valid because [consequence of P]; so I never lose. Repeating makes optimal equal mine."

Filled in for interval scheduling:

"Sort by finish time. Take an optimal solution differing at the first position; my interval finishes no later than theirs; swapping keeps validity because everything after starts later than their finish, which is later than mine; so I never lose."

Three sentences, and it is a complete argument. Practising this template on five problems is worth more than reading proofs of twenty.

Say the sort key and its justification together. "Sort by finish time" is a step. "Sort by finish time *because that leaves the most room for what follows*" is the beginning of a proof. Interviewers hear the difference immediately.

When greedy fails, and how to tell fast

Before proving, spend thirty seconds trying to break it. Counterexample search is faster than proof, and it is the cheaper failure.

The standard probes:

- **Two candidates with the same greedy key but different consequences.** If your key does not distinguish them, the tie-break may be doing unjustified work.
- **A large item that blocks several small ones.** The classic 0/1 knapsack failure: greedy by value density takes an item that excludes two better ones.
- **A locally best move that removes a future option.** Coin change with {1, 3, 4} and target 6: greedy takes 4, then 1, 1 for three coins; optimal is 3 + 3 for two.
- **Boundary sizes.** Empty input, one element, all elements identical, all incompatible.

The coin-change example is worth memorizing because it is the shortest complete refutation of "greedy works for coin change", which candidates assert surprisingly often. Greedy is correct for *canonical* systems such as US coins, and the qualifier is the whole answer.

A failed proof attempt is diagnostic. If you cannot complete the exchange step - if swapping greedy's choice into optimal breaks validity - the place it breaks usually is the counterexample. Proof and refutation are the same search from opposite ends.

Search for the counterexample mechanically

Thirty seconds of thinking is the first pass. If the instance space is small, the second pass is a loop, and it is worth having this shape in your fingers:

JAVA (continued 1/2)

```
import java.util.*;

final class GreedyRefuter {

    /** Fewest coins for `target`, or Integer.MAX_VALUE if unreachable. */
    static int optimalCoins(int[] coins, int target) {
        int[] best = new int[target + 1];
        Arrays.fill(best, Integer.MAX_VALUE);
        best[0] = 0;
        for (int amount = 1; amount <= target; amount++) {
            for (int coin : coins) {
                if (coin <= amount && best[amount - coin] != Integer.MAX_VALUE) {
                    best[amount] = Math.min(best[amount], best[amount - coin] + 1);
                }
            }
        }
        return best[target];
    }

    /** Take the largest coin that fits, repeatedly. */
    static int greedyCoins(int[] coins, int target) {
```

JAVA (continued 2/2)

```

    int[] descending = coins.clone();
    Arrays.sort(descending);
    int used = 0;
    int remaining = target;
    for (int i = descending.length - 1; i >= 0 && remaining > 0; i--) {
        used += remaining / descending[i];
        remaining %= descending[i];
    }
    return remaining == 0 ? used : Integer.MAX_VALUE;
}

/** The smallest target where the two disagree, or -1 if none up to `limit`. */
static int firstDisagreement(int[] coins, int limit) {
    for (int target = 1; target <= limit; target++) {
        if (greedyCoins(coins, target) != optimalCoins(coins, target)) {
            return target;
        }
    }
    return -1;
}
}

```

`firstDisagreement(new int[]{1, 3, 4}, 20)` returns 6: greedy spends 4 + 1 + 1 for three coins where 3 + 3 needs two. That is the entire refutation of "greedy works for coin change", produced without any insight - which is the point. When you cannot see the counterexample, enumerate for it.

The same three-method shape - an obviously-correct oracle, the greedy under test, and a smallest-disagreement search - transfers to every greedy claim you are unsure about. Reach for it *before* the proof, because it is faster and a counterexample ends the question outright.

What a clean run does and does not tell you. `firstDisagreement` returning -1 is evidence, not proof: it says the two agree on the range you searched. Interval scheduling by earliest finish survived 500 exhaustive instances here, and that is still weaker than the exchange argument above it. Use the search to *refute*; use the proof to *accept*.

When greedy is guaranteed: the matroid connection

A structural answer exists for why greedy works on some problems and not others, and naming it is a strong senior signal even without the full theory.

A **matroid** is a set system closed under subsets with an exchange property: if **A** and **B** are independent sets and $|A| < |B|$, then some element of **B** can be added to **A** keeping it independent. **On a matroid, the greedy algorithm is optimal for any linear weight function.** Kruskal's minimum spanning tree is the standard instance - forests of a graph form a matroid, which is why sorting edges by weight and taking whatever does not create a cycle is correct.

The practical use is not to prove matroid-hood in an interview. It is to know that "does this structure have the exchange property?" is the underlying question, and that when it does not - as in 0/1 knapsack, where taking a heavier item can exclude two lighter ones - greedy has no guarantee and dynamic programming is the fallback.

WHY IT WORKS | Worked example: a proof from scratch

Given jobs each with a deadline and a profit, each taking one unit of time, schedule to maximize profit. A job earns its profit only if it finishes by its deadline.

Greedy: sort by profit descending; for each job, schedule it in the latest free slot at or before its deadline; skip if no slot is free.

Exchange proof. Let **OPT** be optimal and **G** greedy's schedule. Consider the highest-profit job where they differ.

*Case 1 - greedy scheduled job **j**, OPT did not.* **OPT** uses some slot at or before **deadline(j)** for a lower-profit job, or leaves it free. Greedy only scheduled **j** because a slot was free at that point in its own construction.

Replacing that lower-profit job with **j** in **OPT** keeps validity - **j** meets its deadline - and does not decrease profit, since **j** has profit at least as high.

*Case 2 - OPT scheduled **j**, greedy did not.* Greedy skipped **j** only when every slot at or before **deadline(j)** was full, and by the profit-descending order every job in those slots has profit at least **profit(j)**. So **OPT** cannot do better by including **j**; removing it and keeping greedy's jobs is no worse.

Both cases move **OPT** one step toward **G** without losing profit. **QED**

Why the latest free slot? Scheduling as late as possible preserves earlier slots for jobs with tighter deadlines. Choosing the earliest free slot breaks the argument and the algorithm: a job with a late deadline occupies an early slot that a tighter job then needs. The scheduling rule and the proof stand or fall together, which is the general lesson.

WATCH OUT | Edge cases and common mistakes

- Presenting greedy with no correctness argument at all.
- Stating the sort key without saying why that key makes exchange work.
- Sorting interval scheduling by start time or duration; neither supports the argument.
- Asserting greedy solves coin change without the canonical-system qualifier.
- Applying greedy to 0/1 knapsack by value density, where a large item blocks two better ones.
- Scheduling into the earliest free slot rather than the latest, breaking the deadline proof.
- Skipping counterexample search and proving something false.
- Abandoning a failed exchange step instead of reading the counterexample out of it.
- Claiming greedy is optimal because it passed the sample cases.
- Confusing "greedy is a good approximation" with "greedy is optimal" - set cover admits a logarithmic-factor greedy, not an exact one.
- Forgetting ties. Two candidates with equal keys may not be interchangeable, and the proof must handle it.

TRY IT | Interview questions and model answers

How do you know your greedy algorithm is correct?

By an exchange argument or by greedy-stays-ahead. Exchange: take any optimal solution, find the first place it differs from mine, and show swapping my choice in keeps it valid and no worse - repeat until optimal becomes mine. Stays-ahead: show my partial solution dominates any other's by some measure at every step, by induction. Which applies depends on the problem; selection problems usually take exchange, coverage problems usually take stays-ahead.

Prove that earliest-finish-time is optimal for interval scheduling.

Exchange. Suppose an optimal solution agrees with greedy on the first k intervals and differs at $k+1$. Greedy's interval finishes no later than optimal's, since it picks the earliest finish among compatible intervals. Swap it in: every later interval in optimal starts after optimal's finish, which is at or after greedy's, so nothing conflicts. Count unchanged, still optimal, now agreeing on $k+1$. Repeat.

Why not sort by start time or by duration?

Because neither supports the exchange step. The argument needs "everything after starts later than the finish time I chose", and only finish time gives that. Concretely, one long interval starting earliest destroys start-time ordering, and two short intervals overlapping one medium one destroys duration ordering.

Does greedy solve coin change?

Only for canonical coin systems, US denominations among them. With $\{1, 3, 4\}$ and target 6, greedy takes 4 then 1 and 1 for three coins, while the optimum is $3 + 3$ for two. For arbitrary denominations the problem needs dynamic programming.

When is greedy guaranteed to work?

When the structure is a matroid - a set system closed under subsets with the exchange property that a smaller independent set can always absorb an element from a larger one. Greedy is then optimal for any linear weight function, which is why Kruskal's MST works. Practically the question is whether that exchange property holds; where it fails, as in 0/1 knapsack, greedy has no guarantee and DP is the fallback.

Your exchange argument does not go through. What does that tell you?

Usually that the algorithm is wrong, and that the step where the swap breaks validity is the counterexample. Proof and refutation are the same search from opposite ends, so a stuck proof is a strong hint about where to look for a failing input.

TRY IT | Exercises

- 1 **Foundation:** Write the exchange argument for interval scheduling in three sentences, then compress it to one.
- 2 **Foundation:** Find counterexamples to interval scheduling sorted by start time and by duration.
- 3 **Interview Core:** Prove earliest-finish-time optimal by greedy-stays-ahead, and compare the two proofs' lengths.
- 4 **Interview Core:** Prove or refute: greedy by value density is optimal for fractional knapsack, then for 0/1 knapsack.
- 5 **Interview Core:** Prove the deadline-scheduling algorithm, then change to the earliest free slot and construct the failing input.
- 6 **Interview Core:** Show $\{1, 3, 4\}$ breaks greedy coin change, then characterize a canonical system.
- 7 **SDE-2 Follow-up:** Prove Kruskal via the exchange argument, then state which matroid property it uses.
- 8 **SDE-2 Follow-up:** Take gas station and prove it by stays-ahead. Say why exchange is awkward here.
- 9 **SDE-2 Follow-up:** Take a greedy that fails, attempt the exchange proof, and extract the counterexample from where it breaks.
- 10 **Challenge:** For a greedy of your choice, write both proofs and state which you would give in an interview and why.

RECAP | Chapter summary

Greedy is the technique where a plausible algorithm is usually wrong, so the correctness argument is not an add-on - it is the answer. Two techniques cover nearly everything: the exchange argument transforms any optimal solution step by step into the greedy one without loss, and greedy-stays-ahead shows by induction that greedy's partial solution dominates any competitor's at every step. Selection problems usually take exchange, coverage problems usually take stays-ahead, and interval scheduling admits both. Under time pressure, three sentences suffice - state the sort key together with the property that makes the swap safe, because "sort by finish time" is a step while "sort by finish time because it leaves the most room for what follows" is the beginning of a proof. Before proving, spend thirty seconds trying to break it: coin change with $\{1, 3, 4\}$ refutes the most commonly asserted false claim, and a proof that will not go through usually hands you the counterexample at the point it fails. The structural answer underneath is the matroid exchange property, which is why Kruskal works and why 0/1 knapsack does not.

TRY IT | Revision checklist

- ☐ I can state both proof techniques and when each applies.
- ☐ I can give the interval-scheduling exchange argument in three sentences.
- ☐ I state the sort key together with the property that justifies it.
- ☐ I can refute start-time and duration ordering with concrete inputs.
- ☐ I search for a counterexample before attempting a proof.
- ☐ I know $\{1, 3, 4\}$ breaks greedy coin change and what "canonical" means.
- ☐ I can prove deadline scheduling and say why the latest free slot is required.
- ☐ I know a stuck exchange step usually locates the counterexample.
- ☐ I can name the matroid exchange property and what it guarantees.
- ☐ I distinguish greedy being optimal from greedy being a bounded approximation.

SDE-2 CORE CHAPTER

Chapter 7 - Greedy Algorithms Practice Lab

- 1 State the greedy rule and exchange proof for activity selection.
- 2 Build a counterexample to shortest-interval-first.
- 3 Explain half-open versus closed interval compatibility.
- 4 Debug a comparator that subtracts interval endpoints.
- 5 Debug fractional-knapsack logic applied to 0/1 items.
- 6 Merge overlapping intervals and explain why this is not activity selection.
- 7 Partition a string so each symbol appears in at most one part.
- 8 Prove or refute a greedy rule for coin change on arbitrary denominations.
- 9 Build Huffman merge cost with a min-heap.
- 10 Schedule jobs to minimize maximum lateness.
- 11 Compare Jump Game reachability and minimum jumps invariants.
- 12 Explain the cut property used by minimum spanning tree algorithms.
- 13 Convert a failed greedy formulation into a DP state.
- 14 Design deterministic tie-breaking without changing optimality.
- 15 Discuss online versus offline scheduling limitations.
- 16 **Interview Core:** Derive the minimum-candy allocation with two directional passes and prove why taking the maximum of the two lower bounds is minimal.

WHY IT WORKS | Proof and counterexample lab

- 17 **Foundation:** Give an exchange proof for earliest-finish interval scheduling and a counterexample to earliest-start.
- 18 **Interview Core:** Return the chosen original interval IDs under half-open semantics and deterministic tie-breaking.
- 19 **Interview Core:** Implement minimum jumps as BFS layers without a queue and return explicit failure for an unreachable suffix.
- 20 **Interview Core:** Prove the gas-station prefix reset and use **long** for cumulative net fuel.
- 21 **Interview Core:** Implement minimum refueling stops with deferred max-heap choices and overflow-safe reach clamping.
- 22 **SDE-2 Follow-up:** Compare interval greedy with exhaustive subsets and jump greedy with a DP oracle on small random inputs; preserve the proof separately.

SDE-2 CORE CHAPTER

Chapter 8 - Greedy Algorithms Practice Lab Solutions

- 1 Choose the compatible interval with earliest finish. Replace the first interval of an optimal schedule with it; later intervals remain feasible, so an optimal schedule using the choice exists.
- 2 Place a short interval across the boundary between two longer but mutually compatible intervals; choosing it blocks both and yields one instead of two.
- 3 $[a, b)$ and $[b, c)$ do not overlap, so `next.start >= previous.end`. Closed intervals share point b and may require strict comparison.
- 4 Use `Comparator.comparingInt(Interval::end).thenComparingInt(Interval::start)`; subtraction can overflow.
- 5 Fractions let density choices be partially taken. Indivisible items can make one high-density item block a better combination; 0/1 knapsack needs DP or another exact method.
- 6 Sort by start and extend the current end on overlap. It computes union coverage groups, not a maximum cardinality compatible subset.
- 7 Record the last index of each symbol. Scan while extending the current partition end; close a partition when the current index reaches that end.
- 8 Standard denominations may be canonical, but arbitrary systems fail. For coins $\{1, 3, 4\}$ and amount 6, greedy gives $4+1+1$ while optimum is $3+3$.
- 9 Repeatedly combine the two smallest weights, add their sum to total, and reinsert. The exchange/coding-tree argument makes smallest-first optimal.
- 10 Sort by increasing deadline. An interchange argument shows swapping an inverted pair does not increase maximum lateness.
- 11 Reachability tracks one farthest reachable boundary. Minimum jumps tracks current and next BFS-layer boundaries and increments only when completing a layer.
- 12 The lightest safe edge crossing a cut can be included in some MST; replacing a heavier crossing edge preserves connectivity without increasing cost.
- 13 Identify the information needed to compare alternative histories, such as best value up to item/index/capacity, then define transitions instead of committing irrevocably.
- 14 Add a secondary key only among choices equivalent under the proof's primary criterion. Confirm it does not change feasibility or objective.
- 15 Offline algorithms see all requests and can sort. Online algorithms need competitive objectives, buffering, preemption, or approximation; exact offline optimality may be impossible with irrevocable decisions.
- 16 Initialize one candy each. A left-to-right pass raises a child's count above a lower-rated left neighbor; a right-to-left pass raises it above a lower-rated right neighbor, taking the maximum with the existing count.

Each direction establishes a necessary lower bound, and their pointwise maximum satisfies both without excess. Time is $O(n)$, auxiliary space $O(n)$, and the total should use `long` for large n .

WHY IT WORKS | Proof and counterexample solutions

- 17 Replace an optimal schedule's first interval with greedy's earliest-finishing interval; it ends no later, so every later compatible interval remains feasible. Repeat. Earliest-start fails for `[0, 100)` together with `[1, 2)`, `[2, 3)`, `[3, 4)`: it selects one instead of three.
- 18 Sort copied records by end, then start, then ID. Select when `start >= lastEnd` under half-open semantics and return original records/IDs. Equal-end tie choice does not reduce maximum count, while deterministic ordering makes tests and downstream explanation stable.
- 19 `currentLayerEnd` bounds indexes reachable in current jump count; scanning them accumulates `farthest` for the next layer. At layer end increment jumps and advance boundary. If scan index exceeds `farthest`, return -1. Compute `index+steps[index]` in long and clamp to destination.
- 20 Track total net and current candidate tank. When tank becomes negative at `i`, no start from the candidate through `i` can cross that failing edge with more fuel, so reset to `i+1`. Total negative proves impossibility; otherwise the final candidate works. Accumulate in long.
- 21 Add every station at/before reachable frontier to a max-heap. When target is still beyond reach, choose largest passed fuel; exchanging it for any smaller chosen refill cannot reduce reach and uses one stop. If heap is empty, fail. Clamp reach at target to prevent overflow and validate sorted station positions.
- 22 Enumerate all subsets for at most ten intervals, sort chosen intervals, and compute maximum compatible count. For jumps, use $O(n^2)$ DP/BFS distance as oracle. Compare seeded random cases. These tests find counterexamples and coding errors but do not replace the exchange/frontier proof over all inputs.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Greedy Proof and Oracle Checks

This dependency-free Java 21 companion validates returned interval choices, minimum-jump frontiers, deferred-refueling heaps, counterexamples, exhaustive interval oracles, and DP jump oracles. A successful run prints PASS 15 greedy checks.

JAVA (continued 1/7)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Random;

public final class GreedyInterviewChecks {
    record Interval(int id, int start, int end) {}

    private GreedyInterviewChecks() {}

    static int minimumRemovals(int[][] input) {
        int[][] intervals = Arrays.stream(input).map(int[][]::clone).toArray(int[][]::new);
        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[1]));
        int kept = 0;
        int lastEnd = Integer.MIN_VALUE;
        for (int[] interval : intervals) {
            if (interval[0] >= lastEnd) {
                kept++;
                lastEnd = interval[1];
            }
        }
        return intervals.length - kept;
    }

    static int startStation(int[] gas, int[] cost) {
        if (gas.length != cost.length || gas.length == 0) {
            throw new IllegalArgumentException("invalid arrays");
        }
        long total = 0L;
        long tank = 0L;
        int start = 0;
        for (int i = 0; i < gas.length; i++) {
            if (gas[i] < 0 || cost[i] < 0) {
                throw new IllegalArgumentException("gas and cost must be nonnegative");
            }
        }
    }
}
```

JAVA (continued 2/7)

```

        }
        long net = (long) gas[i] - cost[i];
        total += net;
        tank += net;
        if (tank < 0L) {
            start = i + 1;
            tank = 0L;
        }
    }
    return total >= 0L ? start % gas.length : -1;
}

/** Maximum-cardinality compatible set for half-open intervals [start,end). */
static List<Interval> selectMaximumNonOverlapping(List<Interval> input) {
    List<Interval> intervals = new ArrayList<>(input);
    for (Interval interval : intervals) {
        if (interval.start() > interval.end()) {
            throw new IllegalArgumentException("interval start exceeds end");
        }
    }
    intervals.sort(Comparator.comparingInt(Interval::end)
        .thenComparingInt(Interval::start)
        .thenComparingInt(Interval::id));
    List<Interval> selected = new ArrayList<>();
    int lastEnd = Integer.MIN_VALUE;
    for (Interval interval : intervals) {
        if (interval.start() >= lastEnd) {
            selected.add(interval);
            lastEnd = interval.end();
        }
    }
    return List.copyOf(selected);
}

static int minimumJumps(int[] maximumSteps) {

```

JAVA (continued 3/7)

```
    for (int step : maximumSteps) {
        if (step < 0) {
            throw new IllegalArgumentException("jump lengths cannot be negative");
        }
    }
    if (maximumSteps.length <= 1) {
        return 0;
    }
    int jumps = 0;
    int currentLayerEnd = 0;
    int farthest = 0;
    for (int index = 0; index < maximumSteps.length - 1; index++) {
        if (index > farthest) {
            return -1;
        }
        long reach = index + (long) maximumSteps[index];
        farthest = (int) Math.min(maximumSteps.length - 1L,
            Math.max(farthest, reach));
        if (index == currentLayerEnd) {
            jumps++;
            currentLayerEnd = farthest;
            if (currentLayerEnd >= maximumSteps.length - 1) {
                return jumps;
            }
        }
    }
    return -1;
}

/** Stations are [position,fuel], sorted by position; fuel can be chosen once. */
static int minimumRefuelStops(long target, long startFuel, long[][] stations) {
    if (target < 0 || startFuel < 0) {
        throw new IllegalArgumentException("target and fuel must be nonnegative");
    }
    long previousPosition = -1;
```

JAVA (continued 4/7)

```
for (long[] station : stations) {
    if (station.length != 2 || station[0] < previousPosition
        || station[0] < 0 || station[0] > target || station[1] < 0) {
        throw new IllegalArgumentException("invalid station sequence");
    }
    previousPosition = station[0];
}
PriorityQueue<Long> passedFuel = new PriorityQueue<>(Comparator.reverseOrder());
long reachable = startFuel;
int stationIndex = 0;
int stops = 0;
while (reachable < target) {
    while (stationIndex < stations.length
        && stations[stationIndex][0] <= reachable) {
        passedFuel.add(stations[stationIndex][1]);
        stationIndex++;
    }
    if (passedFuel.isEmpty()) {
        return -1;
    }
    long fuel = passedFuel.remove();
    reachable = fuel >= target - reachable ? target : reachable + fuel;
    stops++;
}
return stops;
}

private static boolean intervalGreedyMatchesBruteForce() {
    Random random = new Random(61L);
    for (int trial = 0; trial < 1_000; trial++) {
        int count = random.nextInt(11);
        List<Interval> intervals = new ArrayList<>();
        for (int id = 0; id < count; id++) {
            int start = random.nextInt(12);
            int end = start + 1 + random.nextInt(5);
        }
    }
}
```

JAVA (continued 5/7)

```

        intervals.add(new Interval(id, start, end));
    }
    int greedy = selectMaximumNonOverlapping(intervals).size();
    int best = 0;
    for (int mask = 0; mask < 1 << count; mask++) {
        List<Interval> chosen = new ArrayList<>();
        for (int index = 0; index < count; index++) {
            if ((mask & 1 << index) != 0) {
                chosen.add(intervals.get(index));
            }
        }
        chosen.sort(Comparator.comparingInt(Interval::start)
            .thenComparingInt(Interval::end));
        boolean compatible = true;
        for (int index = 1; index < chosen.size(); index++) {
            compatible &= chosen.get(index).start() >= chosen.get(index - 1).end();
        }
        if (compatible) {
            best = Math.max(best, chosen.size());
        }
    }
    if (greedy != best) {
        return false;
    }
}
return true;
}

private static boolean jumpGreedyMatchesDynamicProgramming() {
    Random random = new Random(67L);
    for (int trial = 0; trial < 2_000; trial++) {
        int[] steps = new int[random.nextInt(15)];
        for (int index = 0; index < steps.length; index++) {
            steps[index] = random.nextInt(6);
        }
    }
}

```


JAVA (continued 6/7)

```
int[] best = new int[steps.length];
Arrays.fill(best, Integer.MAX_VALUE);
if (steps.length > 0) {
    best[0] = 0;
}
for (int index = 0; index < steps.length; index++) {
    if (best[index] == Integer.MAX_VALUE) {
        continue;
    }
    int limit = (int) Math.min(steps.length - 1, index + (long) steps[index]);
    for (int next = index + 1; next <= limit; next++) {
        best[next] = Math.min(best[next], best[index] + 1);
    }
}
int expected = steps.length <= 1 ? 0
    : best[steps.length - 1] == Integer.MAX_VALUE
    ? -1 : best[steps.length - 1];
if (minimumJumps(steps) != expected) {
    return false;
}
}
return true;
}

private static void expectFailure(Runnable action) {
    try {
        action.run();
    } catch (IllegalArgumentException expected) {
        return;
    }
    throw new AssertionError("expected IllegalArgumentException");
}

private static void check(boolean condition, String message) {
    if (!condition) throw new AssertionError(message);
}
```

JAVA (continued 7/7)

```

    }

    public static void main(String[] args) {
        check(minimumRemovals(new int[][] {{1, 2}, {2, 3}, {3, 4}, {1, 3}}) == 1,
            "interval removals");
        check(startStation(new int[] {1, 2, 3, 4, 5}, new int[] {3, 4, 5, 1, 2}) == 3,
            "gas station");
        check(startStation(new int[] {2}, new int[] {3}) == -1, "impossible");
        List<Interval> selected = selectMaximumNonOverlapping(List.of(
            new Interval(1, 1, 4), new Interval(2, 3, 5),
            new Interval(3, 0, 6), new Interval(4, 5, 7),
            new Interval(5, 8, 9), new Interval(6, 5, 9)));
        check(selected.stream().map(Interval::id).toList().equals(List.of(1, 4, 5)),
            "interval schedule returns auditable choices");
        check(selectMaximumNonOverlapping(List.of(
            new Interval(1, 1, 2), new Interval(2, 2, 3))).size() == 2,
            "touching half-open intervals are compatible");
        expectFailure(() -> selectMaximumNonOverlapping(
            List.of(new Interval(1, 3, 2))));

        check(minimumJumps(new int[] {2, 3, 1, 1, 4}) == 2, "minimum jumps");
        check(minimumJumps(new int[] {3, 2, 1, 0, 4}) == -1, "unreachable jump");
        check(minimumJumps(new int[0]) == 0, "empty jump contract");

        long[][] stations = {{10, 60}, {20, 30}, {30, 30}, {60, 40}};
        check(minimumRefuelStops(100, 10, stations) == 2, "deferred refuel choices");
        check(minimumRefuelStops(100, 1, stations) == -1, "refuel impossible");
        check(minimumRefuelStops(50, 50, new long[0][0]) == 0, "already reachable");
        expectFailure(() -> minimumRefuelStops(100, 10,
            new long[][] {{20, 1}, {10, 2}}));
        check(intervalGreedyMatchesBruteForce(), "interval exhaustive oracle");
        check(jumpGreedyMatchesDynamicProgramming(), "jump DP oracle");
        System.out.println("PASS 15 greedy checks");
    }
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: activity selection and interval merges
- Interview Core: jumps, gas station, and scheduling
- SDE-2 Follow-up: construct or refute proofs
- Challenge: compare greedy, DP, and graph formulations

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: DSA 15 - Graphs: Traversal, Ordering, Paths, and Union-Find](#)

[Series index](#)

[Next book: DSA 17 - Dynamic Programming: State, Transitions, and Optimization](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 16 of 17 - Study Step DSA-16. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.